Clustering and Initialization: An Experimental Study of K-means and K-means++

Timur Brevnov

May 8, 2026

Abstract

This report studies the effect of initialization on the performance of the K-means clustering algorithm, drawing on the paper *k-means++: The Advantages of Careful Seeding* by Arthur and Vassilvitskii [1]. Both standard K-means and K-means++ were implemented from scratch in Python and evaluated on the PHL Exoplanet Catalog, a publicly available dataset containing physical and astrophysical measurements of confirmed and candidate exoplanets. The experiments compare the two methods across three values of k using inertia, variability across repeated trials, and convergence speed as evaluation criteria. The results consistently show that K-means++ achieves lower average inertia, substantially reduced variance across trials, and faster convergence than standard random initialization, with the advantage becoming especially pronounced at larger values of k.

1 Introduction

One of the most popular clustering algorithms is K-means. Its popularity comes from its simplicity: the algorithm alternates between assigning each data point to its nearest center and recomputing each center as the mean of the points assigned to it. Even though its popular K-means is sensitive to initialization. Because K-means objective is non-convex, the algorithm is only guaranteed to find a local minimum of the inertia, and different starting points can lead to very different results.

The paper *k-means++: The Advantages of Careful Seeding* [1] by Arthur and Vassilvitskii addresses this problem by proposing a randomized initialization procedure, referred to as K-means++, which selects the initial cluster centers probabilistically, biasing the selection towards points that are far from already-chosen centers. Empirically, they also show that k-means++ tends to produce better solutions and to converge in fewer iterations.

The goal of this project is to replicate these experiments on astrophysical data. I implement both algorithms from scratch and conduct a systematic comparison across different values of k, measuring inertia and variability. The dataset I used is the PHL Exoplanet Catalog, which presents practical challenges including high dimensionality, heterogeneous feature scales, and no natural cluster structure to serve as ground truth.

2 Background

2.1 The K-means Objective

The k-means problem is defined as follows. Given a set X of n data points in $\mathbb{R}^d$ and an integer k, the goal is to choose k cluster centers $C = \{c_1, \ldots, c_k\}$ so as to minimize the clustering potential, also called inertia:

$$\phi = \sum_{x \in X} \min_{c \in C} \|x - c\|^2$$

This quantity measures how tightly the data points cluster around their respective centers. Smaller ϕ indicates more compact, well-separated clusters.

2.2 Standard K-means

The K-means algorithm:

1. Randomly pick k initial centers from X.
2. Assign each point to its nearest center.
3. Recompute each center as the mean of all points assigned to it.
4. Repeat the steps 2 and 3 until the centers no longer change significantly.

Each iteration is guaranteed not to increase ϕ, so the algorithm always terminates. In practice, convergence is usually fast, though as outlined before main weakness, is that result depends on initialization. A poor choice of initial centers can trap the algorithm in a bad local minimum.

2.3 K-means++ Initialization

The k-means++ algorithm modifies only the initialization step. The procedure is:

1. Choose the first center c_1 uniformly at random from X.
2. For each subsequent center c_i (with $i = 2, \ldots, k$), let $D(x')$ denote the distance from point x' to the nearest already-chosen center. Sample the next center from X with probability proportional to $D(x')^2$:
$$\Pr[x' \text{ is chosen}] = \frac{D(x')^2}{\sum_{x' \in X} D(x)^2}$$
3. After selecting k centers, proceed with the standard K-means algorithm.

This scheme, referred to as D^2 weighting, favors points that are far from the current set of centers, thereby encouraging the initial centers to be spread across the data.

3 Dataset and Preprocessing

3.1 Dataset Description

The dataset used in this project is the PHL Exoplanet Catalog, maintained by the Planetary Habitability Laboratory at the University of Puerto Rico at Arecibo. The catalog aggregates data from major exoplanet surveys and contains measurements for confirmed and candidate exoplanets along with properties of their host stars. The full dataset includes 112 features.

3.2 Feature Selection

Since K-means operates on numerical features and uses Euclidean distance following variables were selected:

- **Planetary orbital and physical properties**: planetary radius, orbital period, semi-major axis, periastron, and apastron.
- **Radiation and thermal properties**: stellar flux and equilibrium temperature.
- **Stellar properties**: apparent magnitude, distance from Earth, metallicity, stellar mass, stellar radius, effective temperature, surface gravity, and luminosity.
- **Habitability-related porperties**: snow line distance, abiogenesis zone indicator, tidal locking parameter, and the Earth Similarity Index (ESI).

Variables like planet names and detection methods, identifier columns, habitability labels were excluded.

3.3 Data Cleaning and Standardization

Rows containing missing values were removed and after cleaning all features were standardized using the z-score ransformation:

$$z = \frac{x - \mu}{\sigma}$$

where μ is mean and σ is standard deviation.

Standardization is essential for K-means because Euclidean distance is sensitive to feature scales. If not performed features measured on larger scales would dominate the distance, making other features irrelevant.

After all pre-processing I have 2073 instances with 20 variables (n = 2073, d = 20)

4 Implementation

Both algorithms were implemented from scratch in Python, only using NumPy for numerical computation and array operations, Pandas for data loading and preprocessing, and Matplotlib together with Seaborn for producing all visualizations.

5 Experimental Setup

Experiments were conducted for $k \in \{10, 25, 50\}$, following the same range used by the authors of a paper [1]. For each value of k, both methods were run 20 times, its necessary because both initialization strategies are randomized, and a single run gives an incomplete picture of an algorithm's behavior. The random seeds were fixed across the two methods for each trial number, so that trial i of K-means and trial i of K-means++ share the same seed at the start of initialization.

The metrics recorded for each run were:

- **Average inertia**
- **Minimum inertia**
- **Standard deviation of inertia**
- **Average number of iterations**

6 Results

6.1 Summary of Results

Table 1 shows results across all 20 trials for each value of k. The columns report the average inertia per point, the percentage improvement of k-means++ over k-means, and the standard deviation of inertia, which reflects the stability of each method.

k	K-means $\bar{\phi}$	K-means++ $\bar{\phi}$	K-means $\phi_{\min}$	K-means++ $\phi_{\min}$	K-means σ	K-means++ σ
10	17238.26	11.38%	15113.16	8.78%	1293.48	1251.39
25	10853.31	23.86%	8733.99	8.49%	1010.18	397.84
50	7166.73	18.15%	5908.78	2.19%	1051.75	41.04

Table 1: Clustering results over 20 trials. For K-means the actual inertia is shown. For K-means++ the percentage improvement over K-means is shown: $100 \cdot \left(1 - \frac{\text{k-means++ value}}{\text{k-means value}}\right)$. The standard deviation column refers to values of K-means and K-means++.

Several patterns stand out. For $k = 10$, K-means++ achieves an average inertia improvement of approximately 11%. As k increases, the gap in stability becomes much more striking. At $k = 25$, the standard deviation of inertia for K-means is 1010 and 398 for K-means++. At $k = 50$, the contrast is even more dramatic: K-means exhibits a standard deviation of 1052 while K-means++ achieves a standard deviation of just 41.

The improvement in minimum inertia is more modest, particularly at $k = 50$: a difference of about 2.2%. This suggests that standard K-means can, with favorable initialization, occasionally reach solutions competitive with K-means++. The key difference lies in reliability: K-means++ achieves good solutions consistently.

6.2 Distribution of Clustering Outcomes

The boxplots in Figure 1 show the distribution of final inertia values across 20 trials for each method and each value of k.

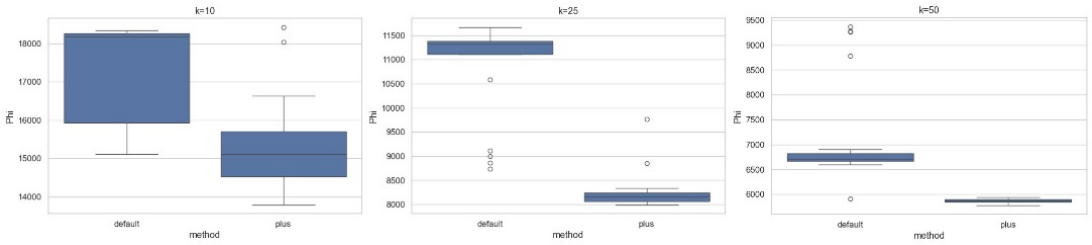

Figure 1: Distribution of inertia values across 20 independent trials for standard K-means and K-means++, shown for each value of k.

The k-means++ box at $k = 50$ is nearly collapsed to a point, consistent with the really small standard deviation reported in Table 1. The medians of K-means++ are uniformly lower than those of K-means across all three values of k, confirming that the improvement is not driven by a few outlier runs but is a consistent effect.

6.3 Stability Comparison

Figure 2 provides a direct comparison of the standard deviation of inertia across the two methods as a function of k.

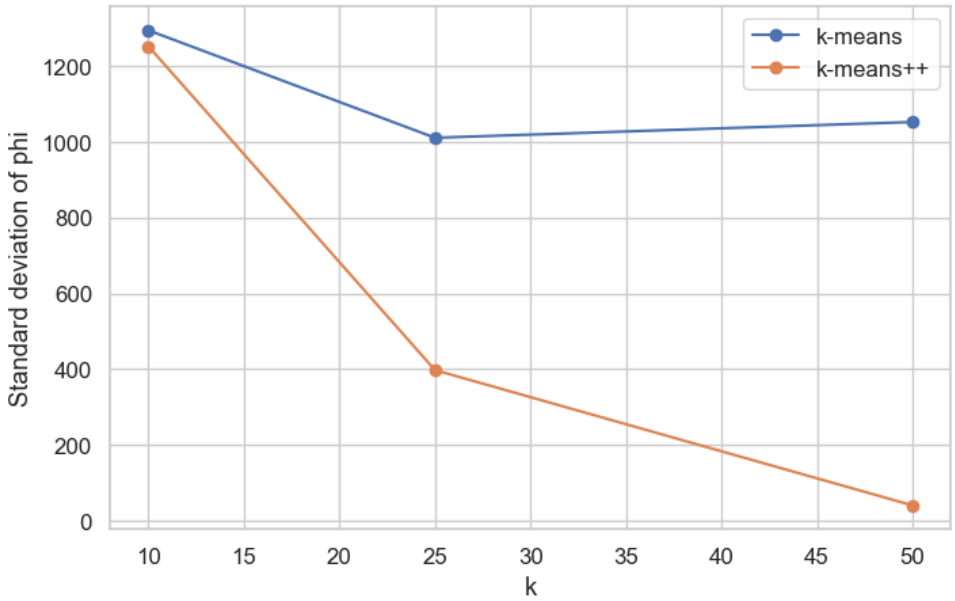

Figure 2: Standard deviation of inertia across 20 trials as a function of k. Lower values indicate more consistent, reliable results across repeated runs.

The figure shows that while the stability of K-means does not improve substantially with k, K-means++ becomes dramatically more stable. At $k = 50$, its standard deviation is approximately 25 times smaller than that of K-means.

6.4 Convergence Behavior

Figure 3 shows the average convergence curves for both methods, i.e., the average inertia at each iteration, averaged over 20 trials.

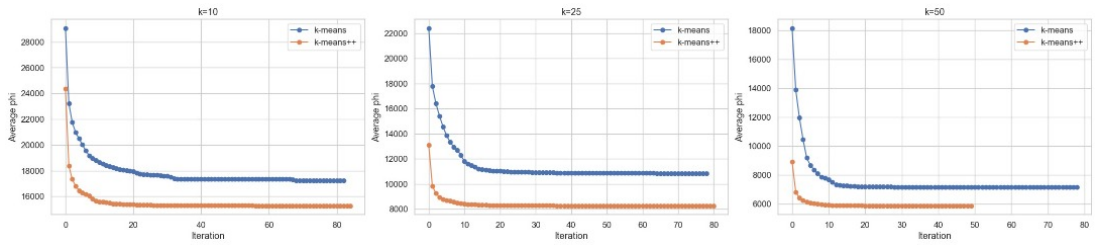

Figure 3: Average inertia per point over iterations, averaged across 20 trials. Curves are shown separately for each value of k. The starting point of each curve reflects the inertia immediately after initialization.

The most notable observation from the convergence curves is that K-means++ starts with lower inertia at iteration zero, before any K-means updates have been applied. This is the direct effect of the initialization of From there, both methods improve inertia in a similar way, but since k-means++ starts from a better position, it also ends at a better one. Table 2 summarizes the average number of iterations required for convergence.

k	K-means	K-means++
10	43.00	38.60
25	46.25	34.90
50	35.00	30.15

Table 2: Average number of iterations to convergence over 20 trials.

K-means++ converges in fewer iterations for all three values of k. The reduction is most substantial for $k = 25$, where K-means++ requires on average 35 iterations versus 46 for k-means, a reduction of roughly 25%.

7 Discussion

7.1 Interpretation of Results

The experimental results confirm the central claim of the K-means++ paper: the initialization step is not a minor implementation detail but a meaningful algorithmic choice that significantly affects both the quality and the reliability of the final clustering. Across all three values of k tested, K-means++ produced lower average inertia and lower variance than standard K-means.

In this project the most striking results were achieved in terms of stability rather than inertia reduction. The improvement in average inertia ranges from roughly 11% to 24% depending on k, which is meaningful but not as huge as in some of the results from original paper. By contrast, the reduction in inertia standard deviation at $k = 50$ is a factor of 25.

7.2 Why Larger k Benefits More

The stabilizing effect of K-means++ becomes stronger as k grows. This can be understood geometrically as larger values of k create more opportunities for poor random initialization, making careful seeding increasingly important The D^2 weighting scheme directly targets this failure mode by penalizing redundant placements.

8 Conclusion

This project implemented and compared standard K-means and K-means++ initialization on the PHL Exoplanet Catalog. Both algorithms were coded from scratch and evaluated over 20 independent trials for each of $k \in \{10, 25, 50\}$.

The results are consistent with the findings of the original paper. K-means++ achieves lower average inertia than standard K-means in all cases, with relative improvements between 11% and 24%. Moreover, K-means++ is substantially more stable: its inertia variance across trials is far smaller and also converges in fewer iterations on average.

References

[1] David Arthur and Sergei Vassilvitskii. *k-means++: The Advantages of Careful Seeding.* Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA), 2007.